

REVIEW AND RATING MANAGEMENT

Meesho – Women’s Fashion E-Commerce Database Management System

1. Objectives

- Create Review and Rating tables.
- Store customer feedback and product ratings.
- Retrieve product review details.
- Calculate average product ratings using aggregate functions.
- Identify highly rated products.
- Generate a product rating report.

2. Database Design

2.1 Review Table

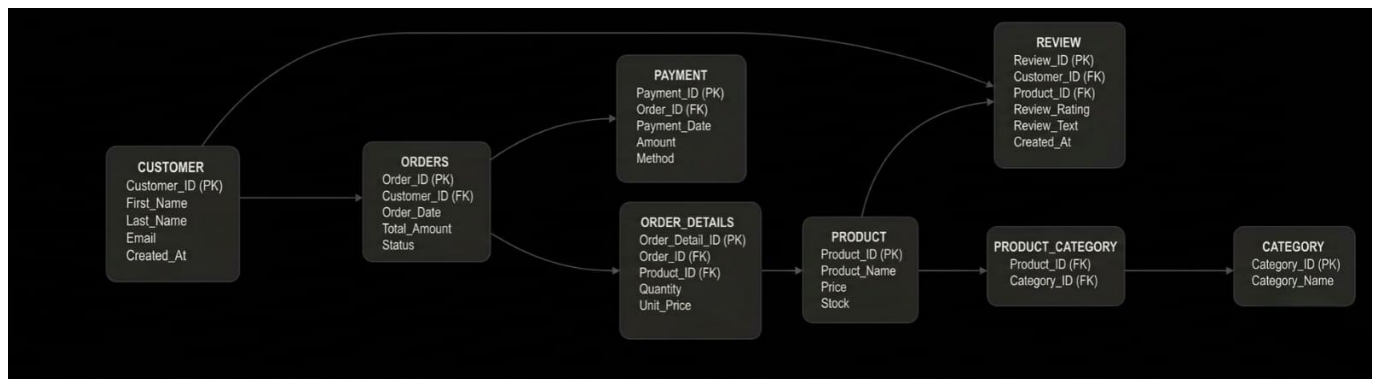
Attribute	Data Type	Constraint
Review_ID	NUMBER	Primary Key
Customer_ID	NUMBER	Foreign Key
Product_ID	NUMBER	Foreign Key
Review_Text	VARCHAR2(500)	NOT NULL
Review_Date	DATE	NOT NULL

2.2 Rating Table

Attribute	Data Type	Constraint
Rating_ID	NUMBER	Primary Key
Customer_ID	NUMBER	Foreign Key
Product_ID	NUMBER	Foreign Key
Rating	NUMBER	NOT NULL
Rating_Date	DATE	NOT NULL

3. Table Relationships

The Review and Rating tables are connected to the existing Meesho Customer and Product tables using Customer_ID and Product_ID foreign keys.



4. Create Review Table

```
SQL> CREATE TABLE Meesho_Reviews (  
 2     Review_ID NUMBER PRIMARY KEY,  
 3     Customer_ID NUMBER,  
 4     Product_ID NUMBER,  
 5     Review_Text VARCHAR2(500),  
 6     Review_Date DATE NOT NULL  
 7 );
```

Table created.

5. Create Rating Table

```
SQL>  
SQL> CREATE TABLE Meesho_Ratings (  
 2     Rating_ID NUMBER PRIMARY KEY,  
 3     Customer_ID NUMBER,  
 4     Product_ID NUMBER,  
 5     Rating NUMBER(1) CHECK (Rating BETWEEN 1 AND 5),  
 6     Rating_Date DATE NOT NULL  
 7 );
```

Table created.

6. Review Data Insertion

The review records should use the Customer_ID and Product_ID values already available in the Meesho database. The following sample follows the existing Meesho data format.

```
SQL> INSERT INTO Meesho_Reviews  
 2 VALUES (1, 101, 101, 'Excellent product and good quality', DATE '2026-10-01');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Meesho_Reviews  
 2 VALUES (2, 102, 102, 'Good product and nice design', DATE '2026-10-02');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Meesho_Reviews  
 2 VALUES (3, 103, 103, 'Average quality', DATE '2026-10-03');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Meesho_Reviews  
 2 VALUES (4, 104, 104, 'Very useful product', DATE '2026-10-04');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Meesho_Reviews  
 2 VALUES (5, 105, 105, 'Quality could be better', DATE '2026-10-05');  
  
1 row created.  
  
SQL>  
SQL> COMMIT;
```

7. Rating Data Insertion

```
SQL> INSERT INTO Meesho_Ratings
  2 VALUES (1, 101, 101, 5, DATE '2026-10-01');

1 row created.

SQL>
SQL> INSERT INTO Meesho_Ratings
  2 VALUES (2, 102, 102, 4, DATE '2026-10-02');

1 row created.

SQL>
SQL> INSERT INTO Meesho_Ratings
  2 VALUES (3, 103, 103, 3, DATE '2026-10-03');

1 row created.

SQL>
SQL> INSERT INTO Meesho_Ratings
  2 VALUES (4, 104, 104, 5, DATE '2026-10-04');

1 row created.

SQL>
SQL> INSERT INTO Meesho_Ratings
  2 VALUES (5, 105, 105, 2, DATE '2026-10-05');

1 row created.

SQL>
SQL> COMMIT;

Commit complete.
```

8. Retrieve Product Review Details

```
SQL> SELECT * FROM Meesho_Reviews;
```

```
REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
```

```
1          101          101
Excellent product and good quality
01-OCT-26
```

```
2          102          102
Good product and nice design
02-OCT-26
```

```
REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
```

```
Average quality
03-OCT-26
```

```
4          104          104
Very useful product
```

```
REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
```

```
REVIEW_DA
-----
04-OCT-26
```

```
5          105          105
Quality could be better
05-OCT-26
```



9. Calculate Average Product Ratings

```
SQL> SELECT Product_ID,  
2         AVG(Rating) AS Average_Rating  
3 FROM Meesho_Ratings  
4 GROUP BY Product_ID;
```

PRODUCT_ID	AVERAGE_RATING
101	5
102	4
103	3
104	5
105	2

10. Identify Highly Rated Products

Products with an average rating of 4 or above can be identified using GROUP BY and HAVING.

```
SQL> SELECT Product_ID,  
2         AVG(Rating) AS Average_Rating  
3 FROM Meesho_Ratings  
4 GROUP BY Product_ID  
5 HAVING AVG(Rating) >= 4;
```

PRODUCT_ID	AVERAGE_RATING
101	5
102	4
104	5

11. Product Rating Report

A combined report can be generated by joining the Rating table with the existing Meesho Product table.

```
SQL> SELECT COUNT(Rating) AS Total_Ratings,  
2         SUM(Rating) AS Total_Rating,  
3         AVG(Rating) AS Average_Rating,  
4         MAX(Rating) AS Highest_Rating,  
5         MIN(Rating) AS Lowest_Rating  
< 6 FROM Meesho_Ratings;
```

TOTAL_RATINGS	TOTAL_RATING	AVERAGE_RATING	HIGHEST_RATING	LOWEST_RATING
5	19	3.8	5	2

12. Conclusion

The Review and Rating Management module successfully stores customer feedback and product ratings in the Meesho Women's Fashion Database. SQL queries using SELECT, AVG(), GROUP BY and HAVING help retrieve review details, calculate average ratings, identify highly rated products and generate useful product rating reports.

.